AEGIS: Efficiency-Guided Multitask Distillation for Joint Weakness and Technical-Debt Detection

Awaiz Ahmed
Al Ain University
United Arab Emirates
202310183@aau.ac.ae

Khubaib Amjad Alam
Al Ain University
United Arab Emirates
Khubaib.alam@aau.ac.ae

***Abstract*—The NLBSE'27 tool competition ranks single multitask models that flag weaknesses and self-admitted technical debt (SATD) in C/C++ functions by a score that combines mean F1 with inference runtime and GFLOPs. An audit of the data shows that the SATD label is a comment-tag rule, that the weakness label of 80% of the functions comes from a CWE-pattern heuristic, and that the official pipeline spends most of its computation on padding while truncating the tags it needs. AEGIS (An Efficiency-Guided Inference System) shortens the input, batches by length, and distills a 12-layer UniXcoder teacher into a 3-layer student whose depth is chosen by the score. On the public test split it raises mean F1 from 0.910 to 0.931 with 13× fewer GFLOPs than the CodeBERT baseline. On a Colab T4, the ranking hardware, it runs in 16.4 s instead of 192.7 s and the competition score rises from 0.730 to 0.942; run through the unmodified official notebook, it keeps a mean F1 of 0.927.**

***Index Terms*—weakness detection, self-admitted technical debt, multitask learning, knowledge distillation, efficient inference**

I. Introduction

The NLBSE'27 tool competition asks for one model that reads a C/C++ function and predicts whether it is associated with a weakness and whether it carries a self-admitted technical debt (SATD) annotation [1]. The score weights mean F1 by 0.6 and inference runtime and GFLOPs, measured with `torch.profiler` over the whole test split on a Colab T4, by 0.2 each. The CodeBERT [2] baseline reaches a mean F1 of 0.910 but uses 2.67M of the 3M-GFLOPs budget (score 0.729).

AEGIS (An Efficiency-Guided Inference System) treats the score as the design objective. Our contributions are (i) an audit of what the labels measure and where the baseline loses F1 and compute; (ii) two pipeline fixes, whitespace normalization and length-sorted batching, which alone lift the unchanged baseline checkpoint from a score of 0.754 to 0.862; and (iii) a 3-layer student, chosen by projected score and trained with teacher-initialized multitask distillation, that also runs unmodified in the official notebook.

Related work. Russo et al. trained a multitask CodeBERT on the same MADE-WIC labels and found no significant gain over single-task models [3]; here multitasking is a competition rule. NLBSE'26 entries distilled CodeBERT into small students with per-label thresholds [4], [5], and EarlyBIRD showed that early encoder layers suffice for code classification [6]. Length-sorted batching is standard [7]; we apply it to the evaluation pipeline.

II. Task, Data, and Scoring

Data. The dataset derives from MADE-WIC [8]: OSPR (Linux, Chromium, Firefox), Devign [9] and Big-Vul [10]. The weakness label is the OR of the Devign, Big-Vul and W annotations, where W is a set of CWE-derived patterns [11]. OSPR has no original vulnerability labels [8], so for its functions, 80% of the data, the weakness label *is* the W heuristic. The SATD label is MAT [12]: a function is positive if a comment contains TODO, FIXME, XXX or HACK. The organizers clang-formatted the functions, removed those over 500 `cl100k` tokens, rebalanced the joint label classes and split 70/10/20 (214,586 training and 30,655 public test functions, which are the organizers' validation split; 61,311 hidden). Prevalence (weakness 32.8%, MAT 1.65%) is thus set by the curation, not by real code bases. Only the `Function` text may be used.

Audit. (1) The tag rule alone reaches MAT F1 0.924 on the training split and 0.935 on the public test split (precision 1.000, recall 0.877). The misses have no tag in the released text; the organizers' curation script[1] reads each function's leading comment but does not write it, which most likely explains them. About 12% of MAT positives are therefore unrecoverable. (2) The official pipeline tokenizes raw functions with CodeBERT and truncates 10.6% of them at 512 tokens; all 39 tagged MAT positives that the baseline misses have their tag beyond token 512. (3) Weakness prevalence rises from 8.5% to 60.3% across length quartiles (a length-only threshold reaches F1 0.603), and 18% of test functions have an identifier-abstracted clone in training. Random splits are known to inflate such scores [13], [14].

Scoring. With $\bar{F}_1$ the mean F1, t the runtime in seconds and G the GFLOPs (totals over the split, averaged over two timed forward passes),

$$S = 0.6\,\bar{F}_1 + 0.2\max\left(\tfrac{1000-t}{1000}, 0\right) + 0.2\max\left(\tfrac{3\cdot 10^6 - G}{3\cdot 10^6}, 0\right). \quad (1)$$

One F1 point (0.01) is worth 0.006 score, as much as 30 s or 90,000 GFLOPs. The profiler counts only matrix multiplications, so an encoder with L layers, width d and feed-forward size d_{ff} costs

$$G \approx 10^{-9} \textstyle\sum_b B_b W_b\, L\,(8d^2 + 4d\,d_{ff}) \quad (2)$$

[1] https://github.com/moritzmock/NLBSE2027-setup (curate_dataset.py)

for batches of B_b sequences padded to width W_b (within 0.04% of every measurement here). Fused attention is not counted (it would add 11% for the baseline and 5.5% for AEGIS). The notebook pads the whole split to 512 tokens, so only 43% of processed positions are real tokens.

III. The AEGIS Approach

Input compaction. We collapse every whitespace run to one space and use UniXcoder's tokenizer [15]. Mean input length drops from 235 (CodeBERT, raw) to 154 tokens and truncation from 10.6% to 0.5%; comments, and thus the tags, are kept. **Length-sorted batching.** Functions are sorted by length, batched by 600 and padded to each batch's maximum; predictions are unchanged. The rules allow any test-time preprocessing except sampling. **Score-guided depth.** The student keeps the embeddings and the first k layers of the encoder with the standard two-output head, so it is a plain `RobertaForSequenceClassification`. We choose k by the projected score on internal validation. **Teacher-initialized multitask distillation.** A 12-layer UniXcoder teacher is fine-tuned first. The student, initialized from the teacher's first k layers, minimizes

$$\mathcal{L} = \tfrac{\alpha}{2} \sum_t \mathrm{BCE}(z_t, y_t) + \tfrac{1-\alpha}{2} T^2 \sum_t \mathrm{BCE}\big(z_t/T, \sigma(z_t^{\mathrm{T}}/T)\big) \quad (3)$$

over $t \in \{\text{weakness}, \text{MAT}\}$ with student and teacher logits z_t, z_t^{T}, $\alpha = 0.5$, $T = 2$ [16]. **Robustness to the evaluation procedure.** Whether the organizers run participants' inference code is not specified. The final student is trained on a seeded 50/50 mix of raw and normalized functions, and its per-task thresholds τ_t, tuned on internal validation [17], are folded into the output bias ($b_t \leftarrow b_t - \mathrm{logit}(\tau_t)$), so a 0.5 cut-off reproduces them.

IV. Experimental Setup

A stratified 10% of the training split (21,243 functions; exact duplicates moved out) is held out as internal validation; models are fit on the remaining 90% and not retrained. Selection, early stopping (evaluated every half epoch, patience 2) and threshold tuning use only internal validation, so validation scores are slightly optimistic. Training uses AdamW (learning rate $2{\cdot}10^{-5}$ teacher, $5 \cdot 10^{-5}$ students; weight decay 0.01), batch 32, 6% warmup with linear decay, bf16 and seed 42; the teacher trains 3 epochs, students 4. Cost is measured with the official loop (two timed passes per batch, fp16, `torch.profiler`), re-implemented and checked against the official notebook, on an RTX 5080 Laptop GPU, and for the final comparison on a Colab T4 with the released notebook. Each block of Table I was measured in one session; no public-test result informed any decision (an accidental early test run, documented in the repository, was discarded). GFLOPs do not depend on the hardware; runtimes do.

V. Results

Main results. On the T4, AEGIS scores 0.942 against 0.730 for the baseline (16.4 s vs. 192.7 s), with the same predictions as on our GPU. AEGIS improves mean F1 over the baseline

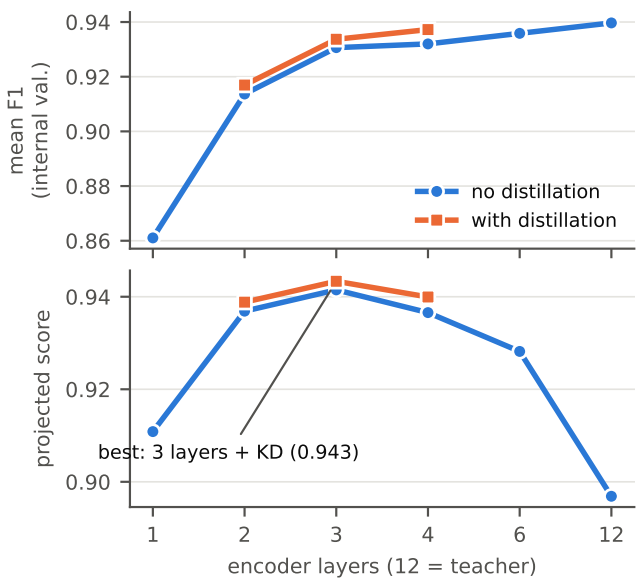


Fig. 1. Encoder depth vs. internal-validation mean F1 (top) and projected score (bottom). Points at 3 layers average three seeds.

by +0.021 (paired bootstrap 95% CI $[0.014, 0.029]$) with 13× fewer GFLOPs, 2.3× from padding and 5.7× from the smaller model and shorter inputs, and runs 11× faster on our GPU. The gain is in MAT (+0.049, $[0.035, 0.066]$): the baseline misses 39 MAT positives whose tag is in the function, all because the tag lies beyond token 512; AEGIS misses 1. On weakness AEGIS is 0.007 below the baseline ($[-0.010, -0.003]$). Batching alone lifts the unchanged baseline to 0.862. The 12-layer teacher is the most accurate model but scores 0.904. The single-line student scores slightly higher (0.945); mixed-input training costs 0.004 weakness F1 ($[-0.006, -0.001]$) in exchange for working in the unmodified notebook, where AEGIS keeps a mean F1 of 0.927 (a verbatim replica of the notebook gives identical predictions).

Depth and distillation. F1 grows with depth, but the projected score peaks at 3 layers (Fig. 1, Table II); the optimum is flat: totalled over the twice larger hidden split without rescaled constants, 2 layers would lead by ≈0.001 (0.9275 vs. 0.9264). All three distilled 3-layer seeds (0.9335–0.9338) exceed all three undistilled ones (0.9297–0.9311) on internal validation, and the test split confirms the weakness gain (+0.006, $[0.004, 0.009]$). A control that initializes from the teacher without distillation reaches 0.9302–0.9317 (3 seeds): of the weakness gain, +0.0017 comes from the initialization and +0.0047 (73%) from the distillation loss. MAT is unchanged by distillation; from two layers on, every model learns the tag rule. Tuned thresholds changed public-test mean F1 by at most 0.0004 for the students, so their benefit is robustness to a 0.5 cut-off rather than accuracy.

Error analysis. 61 of the 62 MAT false negatives have no tag in the released function, and AEGIS agrees with the tag rule on 30,649 of 30,655 functions (it adds 4 false positives).

TABLE I

PUBLIC-TEST RESULTS (30,655 FUNCTIONS) WITH THE OFFICIAL MEASUREMENT LOOP (TWO TIMED FORWARD PASSES PER BATCH, FP16, `TORCH.PROFILER`); RUNTIME AND GFLOPS ARE TOTALS OVER THE SPLIT. GFLOPS DO NOT DEPEND ON THE HARDWARE; RUNTIMES AND SCORES ARE COMPARABLE ONLY WITHIN A BLOCK. THE ORGANIZERS REPORT 198.9 S AND A SCORE OF 0.729 FOR THE BASELINE ON THEIR T4. THE LAST ROW RUNS AEGIS THROUGH THE UNMODIFIED NOTEBOOK PROCEDURE (RAW INPUT, PADDING TO 512, THRESHOLD 0.5).

Model	Layers	Weak. F1	MAT F1	Mean F1	Runtime (s)	GFLOPs	Score
Colab T4 (official ranking hardware), one session							
CodeBERT baseline, official notebook	12	0.939	0.881	0.910	192.7	2,667k	0.730
AEGIS	3	0.932	0.931	0.931	16.4	206k	0.942
RTX 5080 Laptop GPU, one session							
CodeBERT baseline, official loop	12	0.939	0.881	0.910	71.0	2,666k	0.754
+ sorted dynamic padding	12	0.939	0.881	0.910	30.5	1,172k	0.862
UniXcoder teacher	12	0.949	0.928	0.938	21.5	824k	0.904
Student, no distillation	3	0.929	0.927	0.928	6.0	206k	0.942
Student + KD, single-line input	3	0.936	0.930	0.933	6.0	206k	0.945
AEGIS (KD, mixed input)	3	0.932	0.931	0.931	6.2	206k	0.944
AEGIS in the unmodified notebook	3	0.935	0.918	0.927	19.0	667k	0.908

TABLE II

INTERNAL-VALIDATION ABLATIONS. F1 USES VALIDATION-TUNED THRESHOLDS; COST (GFLOPS) IS MEASURED WITH THE OFFICIAL LOOP ON INTERNAL VALIDATION AND THE SCORE PROJECTS IT TO THE PUBLIC-TEST SIZE. (N): F1 AVERAGED OVER N SEEDS; THE LARGEST SEED STANDARD DEVIATION OF MEAN F1 IS 0.0008. COSTS COME FROM EACH CONFIGURATION'S SEED-42 RUN, MEASURED IN A LOWER GPU POWER MODE THAN TABLE I.

Model	Weak. F1	MAT F1	Mean F1	GFLOPs	Score
1 layer	0.805	0.917	0.861	48k	0.911
2 layers	0.901	0.927	0.914	97k	0.937
3 layers (3)	0.930	0.931	0.931	145k	0.941
4 layers	0.933	0.931	0.932	193k	0.937
6 layers	0.942	0.930	0.936	290k	0.928
12 layers (teacher)	0.948	0.931	0.940	580k	0.897
2 layers + KD	0.908	0.925	0.917	97k	0.939
3 layers + KD (3)	0.937	0.931	0.934	145k	0.943
4 layers + KD	0.943	0.931	0.937	193k	0.940
2L + KD, pretrained init	0.907	0.928	0.917	97k	0.939
3L + KD, 6 epochs	0.938	0.930	0.934	145k	0.943
3L + KD, raw input (2)	0.942	0.924	0.933	169k	0.940
3L + KD, line breaks kept	0.940	0.929	0.935	159k	0.942
3L + KD, mixed input (final) (3)	0.935	0.931	0.933	145k	0.943
3L, teacher init, no KD (3)	0.932	0.930	0.931	145k	0.942
4×256 from scratch	0.803	0.916	0.859	22k	0.913

Weakness F1 depends on the label source: 0.954 on OSPR (W heuristic), 0.866 on Big-Vul and 0.768 on Devign, whose labels come from vulnerability-fixing commits (miss rates 0.05, 0.20, 0.35). Functions with a clone in training are missed less often (0.060 vs. 0.094).

VI. LIMITATIONS

High F1 here largely means reproducing the W heuristic and the MAT tag rule. SATD written without a tag is not learned, Big-Vul and Devign labels are noisy [13], [14], prevalence is set by the curation, and the random split with 18% clones overestimates performance on new projects. AEGIS is not a security tool. Our T4 numbers come from one Colab session; the organizers' T4 measured the baseline at 198.9 s rather than 192.7 s. Seeds vary only the student, and we studied one encoder family and one distillation loss.

VII. CONCLUSION

Under an efficiency-weighted ranking, reading the score paid off most: removing padding and truncation, then choosing depth by projected score. Teacher-initialized distillation recovers about a third of the F1 gap to the 12-layer teacher at a quarter of its cost. Code, logs, configurations and the model are at https://github.com/4waiz/AEGIS and https://huggingface.co/4waiz/aegis-nlbse27.

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) to implement the experiment code, run the experiments, analyze the results and draft this manuscript. All numbers are generated by the released scripts from the logged experiment outputs; the authors are responsible for the content of the paper.

REFERENCES

[1] M. Mock, T. Borsani, B. Russo, F. Santos, J. Penney, P. Valenzuela-Toledo, T. Kehrer, and S. Panichella, "The NLBSE'27 tool competition and challenge," Proceedings of the 6th International Workshop on Natural Language-based Software Engineering (NLBSE'27), to appear, 2027.

[2] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[3] B. Russo, J. Melegati, and M. Mock, "Leveraging multi-task learning to improve the detection of SATD and vulnerability," in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, 2025, pp. 1–12.

[4] P. S. U. Shah, A. Ashfaq, A. Fasseh, and S. D. R. Shah, "TraCC: Efficient multi-label code comment classification through knowledge distillation and adaptive thresholding," in *Proceedings of the 5th International Workshop on Natural Language-based Software Engineering (NLBSE)*, 2026, pp. 33–37.

[5] M. A. Majeed and A. B. Asim, "Lightweight knowledge distillation for multi-label code comment classification," in *Proceedings of the 5th International Workshop on Natural Language-based Software Engineering (NLBSE)*, 2026, pp. 25–28.

[6] A. Grishina, M. Hort, and L. Moonen, "The EarlyBIRD catches the bug: On exploiting early layers of encoder models for more efficient code classification," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023, pp. 895–907.

[7] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, and M. Auli, "fairseq: A fast, extensible toolkit for sequence modeling," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, 2019, pp. 48–53.

[8] M. Mock, J. Melegati, M. Kretschmann, N. E. Diaz Ferreyra, and B. Russo, "MADE-WIC: Multiple annotated datasets for exploring weaknesses in code," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2024, pp. 2346–2349.

[9] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[10] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020, pp. 508–512.

[11] B. Russo, M. Camilli, and M. Mock, "WeakSATD: Detecting weak self-admitted technical debt," in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 448–453.

[12] Z. Guo, S. Liu, J. Liu, Y. Li, L. Chen, H. Lu, and Y. Zhou, "How far have we progressed in identifying self-admitted technical debts? A comprehensive empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 4, pp. 1–56, 2021.

[13] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 121–133.

[14] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1729–1741.

[15] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 7212–7225.

[16] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *NIPS 2014 Deep Learning Workshop*, 2015, arXiv:1503.02531.

[17] Z. C. Lipton, C. Elkan, and B. Naryanaswamy, "Optimal thresholding of classifiers to maximize F1 measure," in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, ser. Lecture Notes in Computer Science. Springer, 2014, pp. 225–239.